

Laboratório de Concorrência · Aula 10

henrique kendi ikeda · RA 240394562 · Turma A

Gerado em 09/10/2026, 21:05:46 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação: uma tarefa espera a outra fazer algo específico antes de continuar. Exemplo: o garçom só serve o prato depois que o cozinheiro termina de prepará-lo.

Competição: várias tarefas disputam o mesmo recurso, e só uma pode usá-lo por vez. Exemplo: várias pessoas querendo usar o mesmo banheiro; uma entra, as outras esperam.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2000000

1ª execução (n = 1.000.000)

1147832

2ª execução (n = 1.000.000)

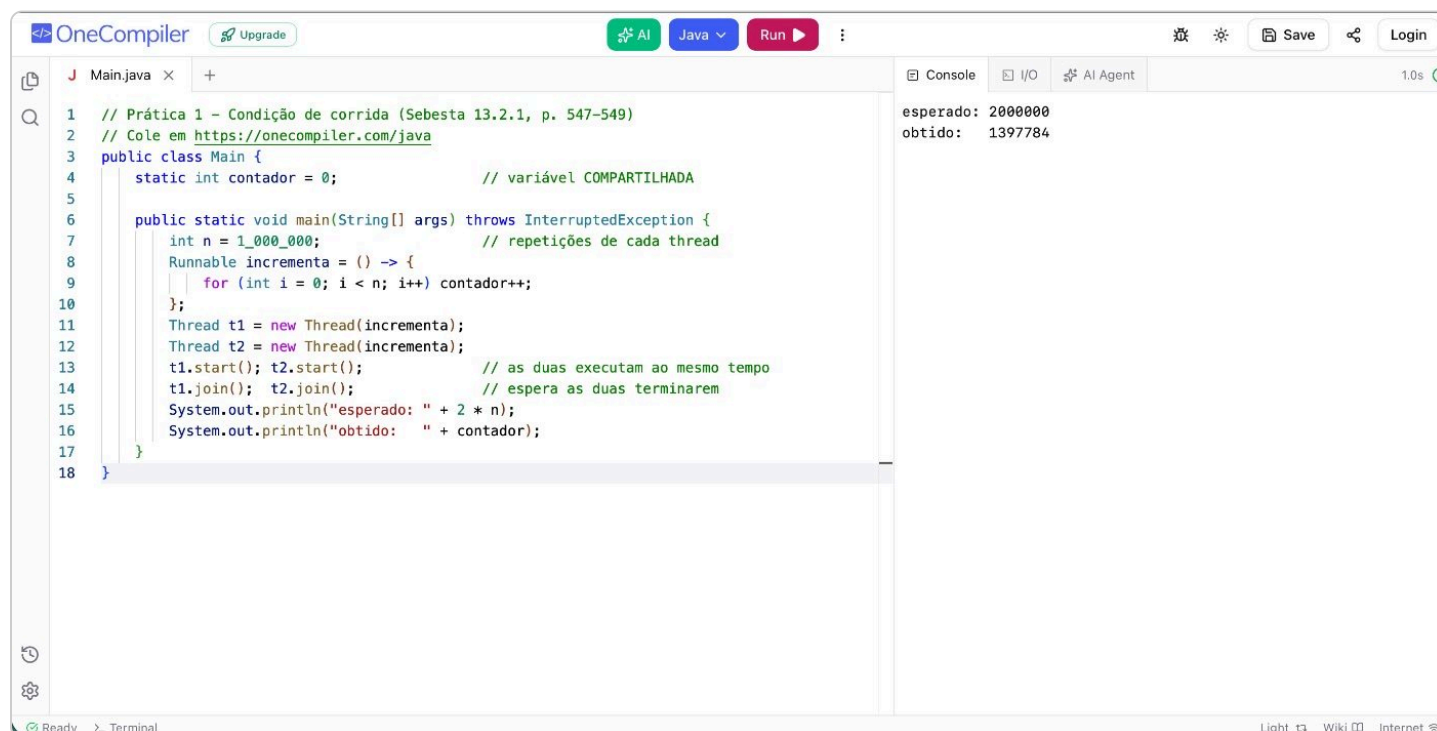
1362095

3ª execução (n = 1.000.000)

1089471

Saída do programa

996671

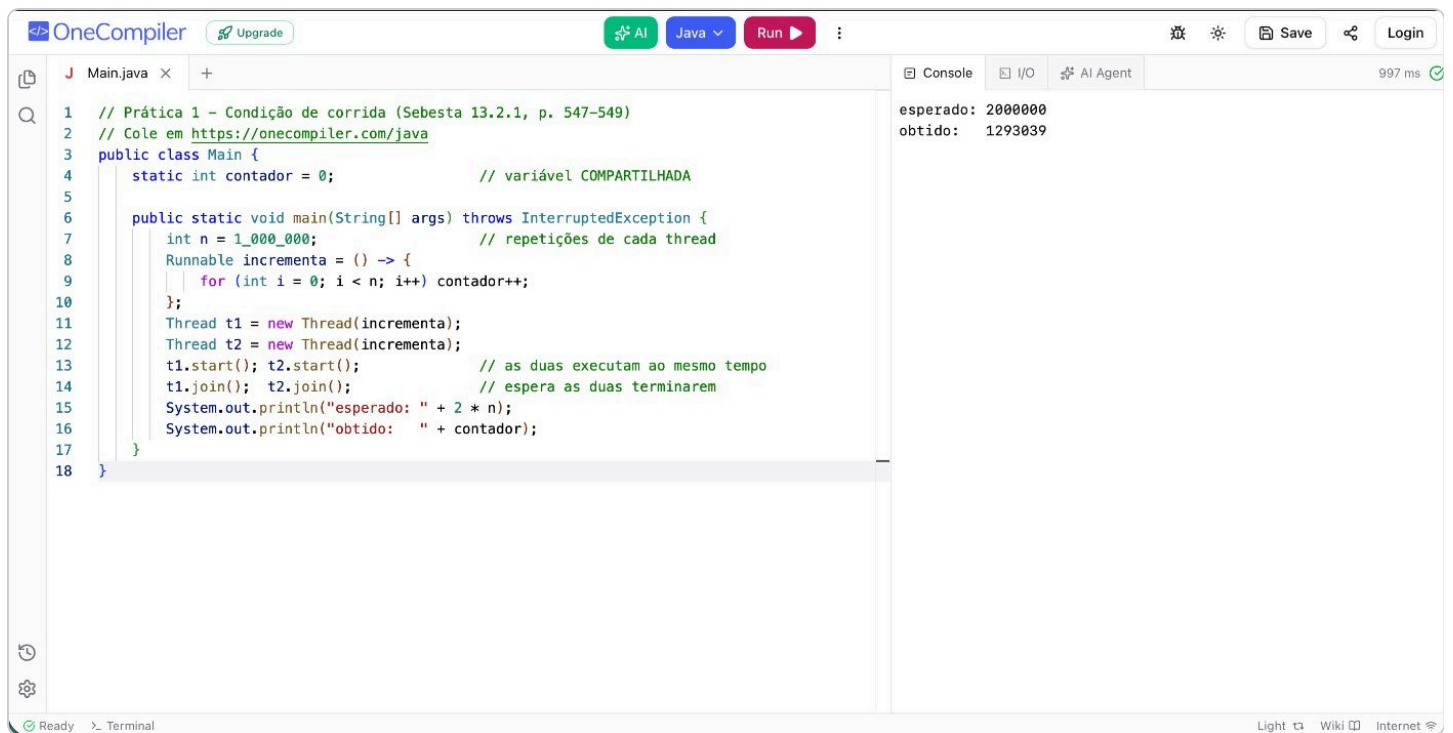


```
// Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
// Cole em https://onecompiler.com/java
public class Main {
    static int contador = 0; // variável COMPARTILHADA

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000; // repetições de cada thread
        Runnable incrementa = () -> {
            for (int i = 0; i < n; i++) contador++;
        };
        Thread t1 = new Thread(incrementa);
        Thread t2 = new Thread(incrementa);
        t1.start(); t2.start(); // as duas executam ao mesmo tempo
        t1.join(); t2.join(); // espera as duas terminarem
        System.out.println("esperado: " + 2 * n);
        System.out.println("obtido: " + contador);
    }
}
```

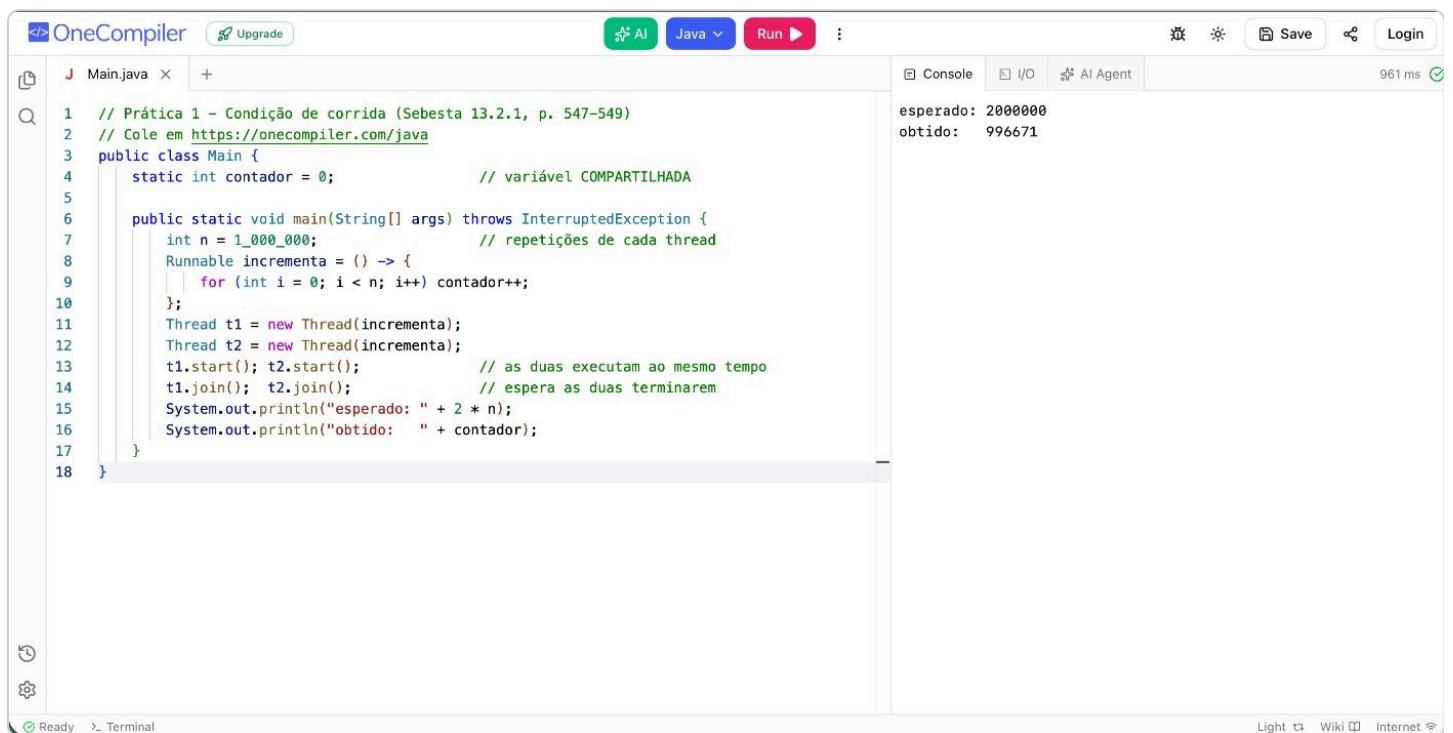
Console

esperado: 2000000
obtido: 1397784



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

esperado: 2000000
obtido: 1293839



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

esperado: 2000000
obtido: 996671

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não. Isso porque, dependendo da velocidade de cada thread, uma delas pode terminar a soma inteira antes de a outra ler o valor do contador, e aí a condição de corrida não aparece.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Thread 1 lê, Thread 1 soma, Thread 2 lê, Thread 1 grava, Thread 2 soma, Thread 2 grava.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

O resultado obtido foi igual ao esperado. Isso é perigoso porque pode dar a falsa impressão de que está tudo certo quando testado em pequena escala, mas, ao aumentar a quantidade de operações, o valor final começa a falhar.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

não respondido

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABAABB → 6
BAAABB → 6
ABBBAA → 4
BBABAA → 4
```

The screenshot shows the OneCompiler IDE with a Python file named `main.py`. The code defines a function `executa(ordem)` that simulates a process with two tasks, A and B, each with a register and a step counter. The function calculates the total value after executing the given order of tasks. The main part of the code generates all possible permutations of 3 steps of task A and 3 steps of task B, calculates the total for each, and prints the results sorted by total value.

```
def executa(ordem):
    total = 3
    reg = {"A": 0, "B": 0}
    passo = {"A": 0, "B": 0}
    for t in ordem:
        if passo[t] == 0:
            reg[t] = total
        elif passo[t] == 1:
            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
        else:
            total = reg[t]
            passo[t] += 1
    return total

print("AAABBB ->", executa("AAABBB"))
print("BBBAAA ->", executa("BBBAAA"))
print("ABABAB ->", executa("ABABAB"))

resultados = {}
for posicoes in combinations(range(6), 3):
    ordem = "".join("A" if i in posicoes else "B" for i in range(6))
    resultados.setdefault(executa(ordem), []).append(ordem)

print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
for valor in sorted(resultados):
    ordens = resultados[valor]
    print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
```

The console output shows the results of the simulation:

```
AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

20 ordens que gera 4 resultados

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Primeiro A lê o valor de total, depois B lê o valor de total. Então B finaliza a execução e salva o valor como 6 e depois B finaliza a execução e salva o valor como 4 (tinha lido 3 e salvou + 1).

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a mais rápida seria o monitor, já que os dados e as operações ficam reunidos no mesmo lugar. O semáforo seria a opção mais problemática, pois pode acontecer de A ficar aguardando para acessar o recurso

enquanto o semáforo está liberado para B, que nem precisa usá-lo.

Saída do programa

não respondido

The screenshot shows the OneCompiler IDE with a Java file named Main.java. The code implements a concurrent program with three threads: 'esperado', 'sem sincronizacao', and 'synchronized'. The 'sem sincronizacao' thread uses a semaphore (semCuidado) and increments it without synchronization. The 'synchronized' thread uses a synchronized block to increment the semaphore. The 'esperado' thread uses an AtomicInteger to increment a counter. The main method runs these threads and prints the results.

```

1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-5)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0, comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
21         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }

```

The console output shows the following results:

```

esperado: 2000000
sem sincronizacao: 1033044 em 2 ms
synchronized: 2000000 em 104 ms
AtomicInteger: 2000000 em 25 ms

```

The screenshot shows the OneCompiler IDE with the same Java code as above. The console output shows the following results:

```

esperado: 2000000
sem sincronizacao: 1478974 em 2 ms
synchronized: 2000000 em 161 ms
AtomicInteger: 2000000 em 29 ms

```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

A versão sem sincronização. Isso porque as execuções e o acesso ao recurso não acontecem de forma simultânea.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve, porque é uma execução composta por duas transações (débito e crédito). Assim, se houver alguma falha no crédito, o dinheiro simplesmente desaparece. Usar o synchronized resolveria o problema, já que garante

que a execução de cada uma dessas transações aconteça de forma sincronizada entre si.

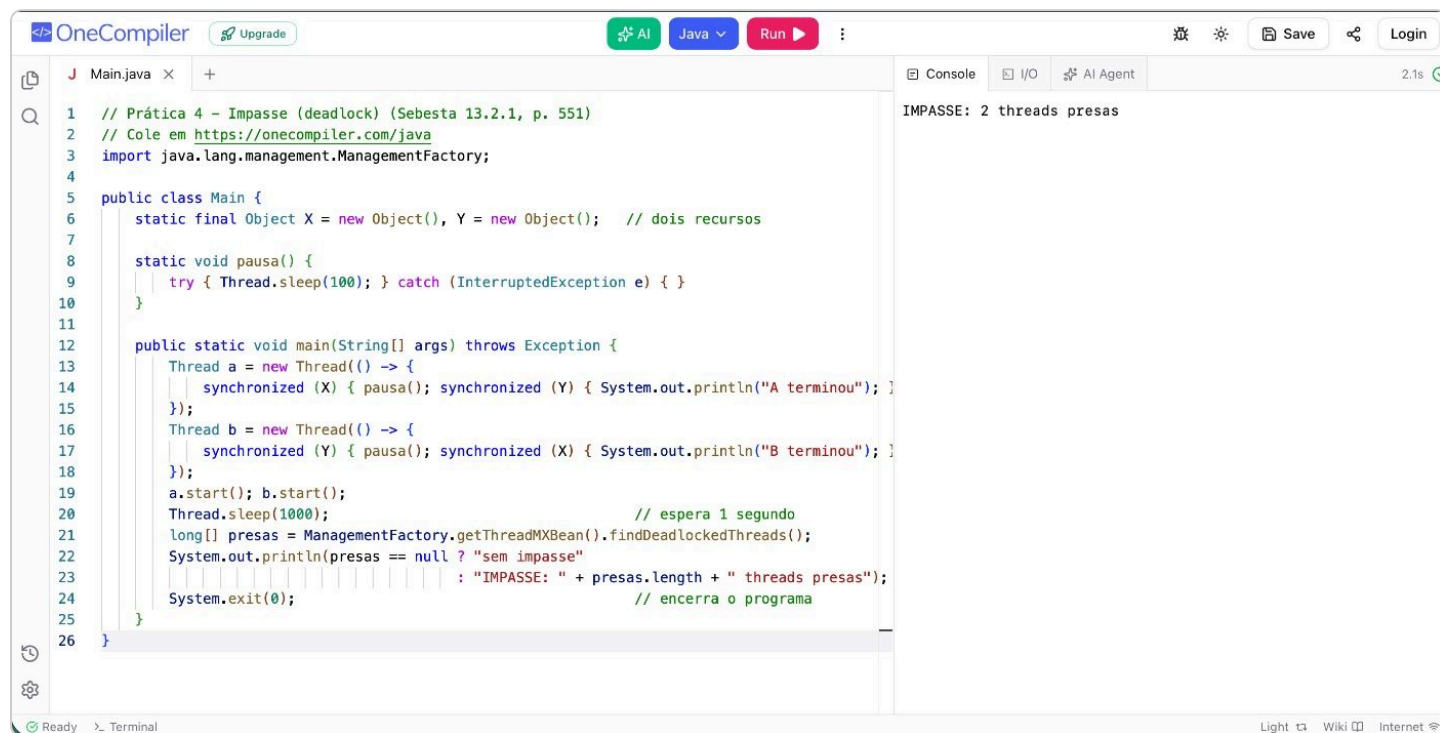
Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

sim

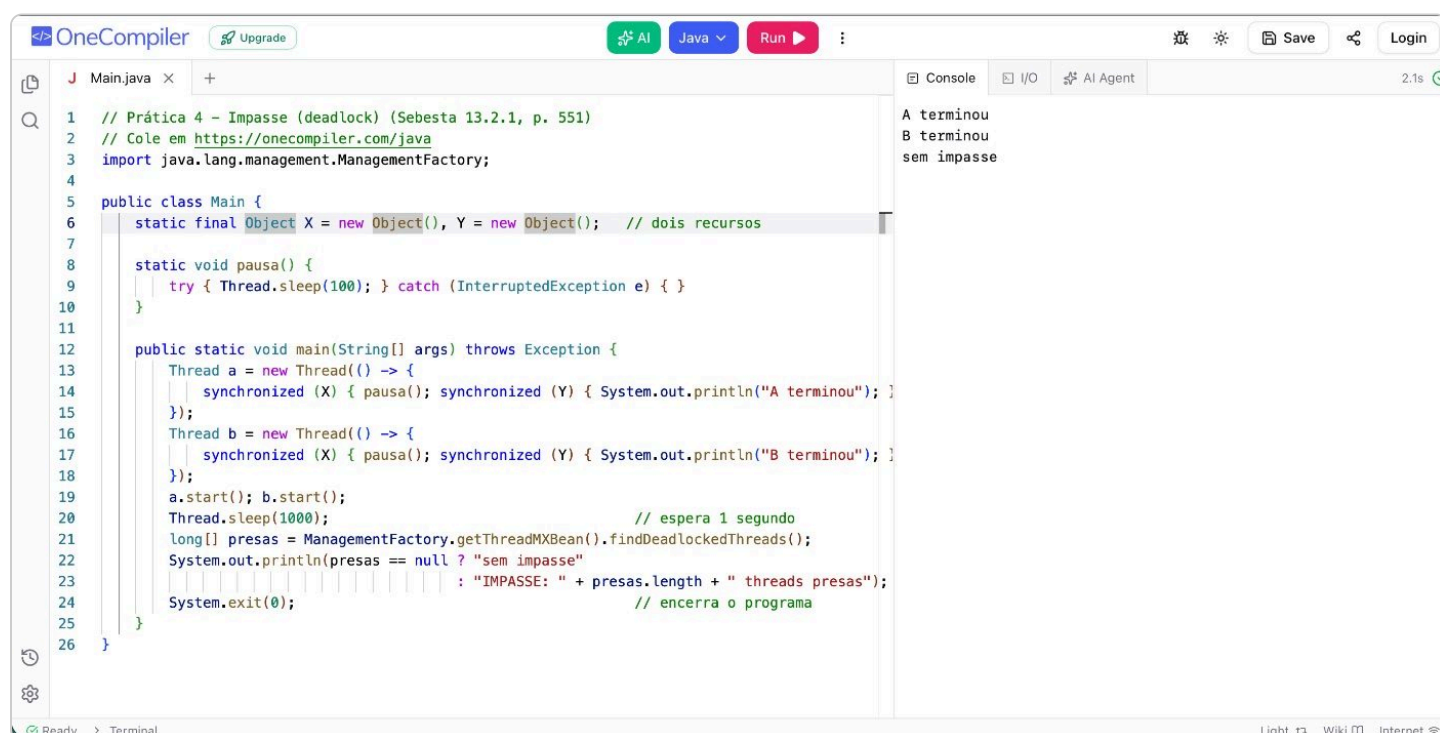
Saídas (antes e depois da correção)

não respondido



```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }
```

IMPASSE: 2 threads presas



```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (X) { pausa(); synchronized (Y) { System.out.println("B terminou"); }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }
```

A terminou
B terminou
sem impasse

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

Porque os recursos necessários para executar a ação estavam sendo usados pela outra thread, então ficaram inacessíveis e as duas entraram em espera. A aguardava Y enquanto B aguardava X.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Porque, quando B tenta obter X, esse recurso já está em uso por Y, e B entra em espera. Isso pode acontecer em um sistema de transações bancárias, em que uma operação tenta acessar o saldo da própria conta enquanto outra transação, vinda de uma conta diferente, tenta acessar o saldo da primeira.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Espero que com 200ms. Com 600ms. Com 100ms

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

não respondido

```
main.go x
5 import (
9 )
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

Console

```
trabalhador 3 fez a tarefa 1
trabalhador 2 fez a tarefa 3
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 6
trabalhador 2 fez a tarefa 5
trabalhador 3 fez a tarefa 4
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

443 ms

Ready Light Wiki Internet

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 55)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 1 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {

```

Console Output:

```

trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

```

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 55)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {

```

Console Output:

```

trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 3 fez a tarefa 4
trabalhador 5 fez a tarefa 6
trabalhador 4 fez a tarefa 5
trabalhador 1 fez a tarefa 2
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

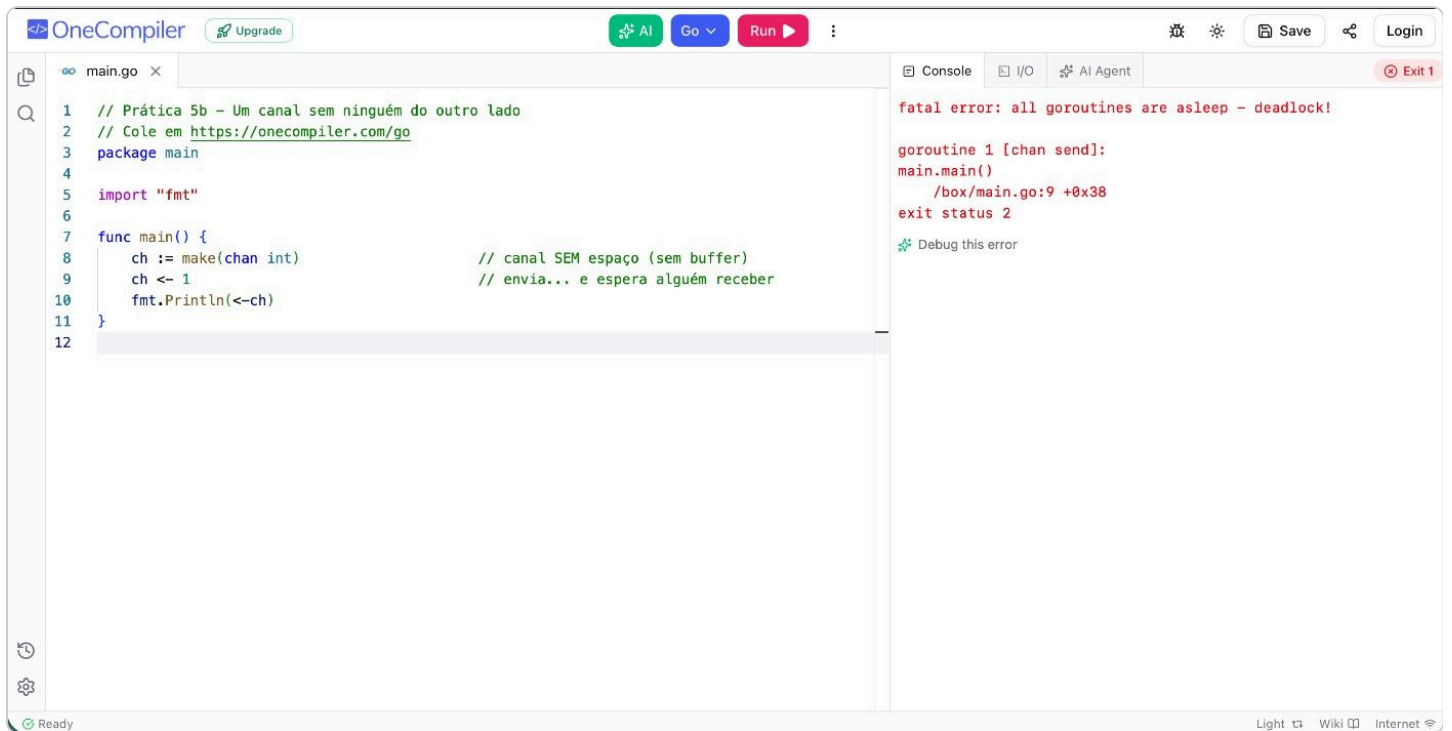
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Porque o trabalhador não acessa as tarefas diretamente, já que existe um "orquestrador" responsável por atribuir uma tarefa a cada trabalhador.

Saídas (antes e depois da troca)

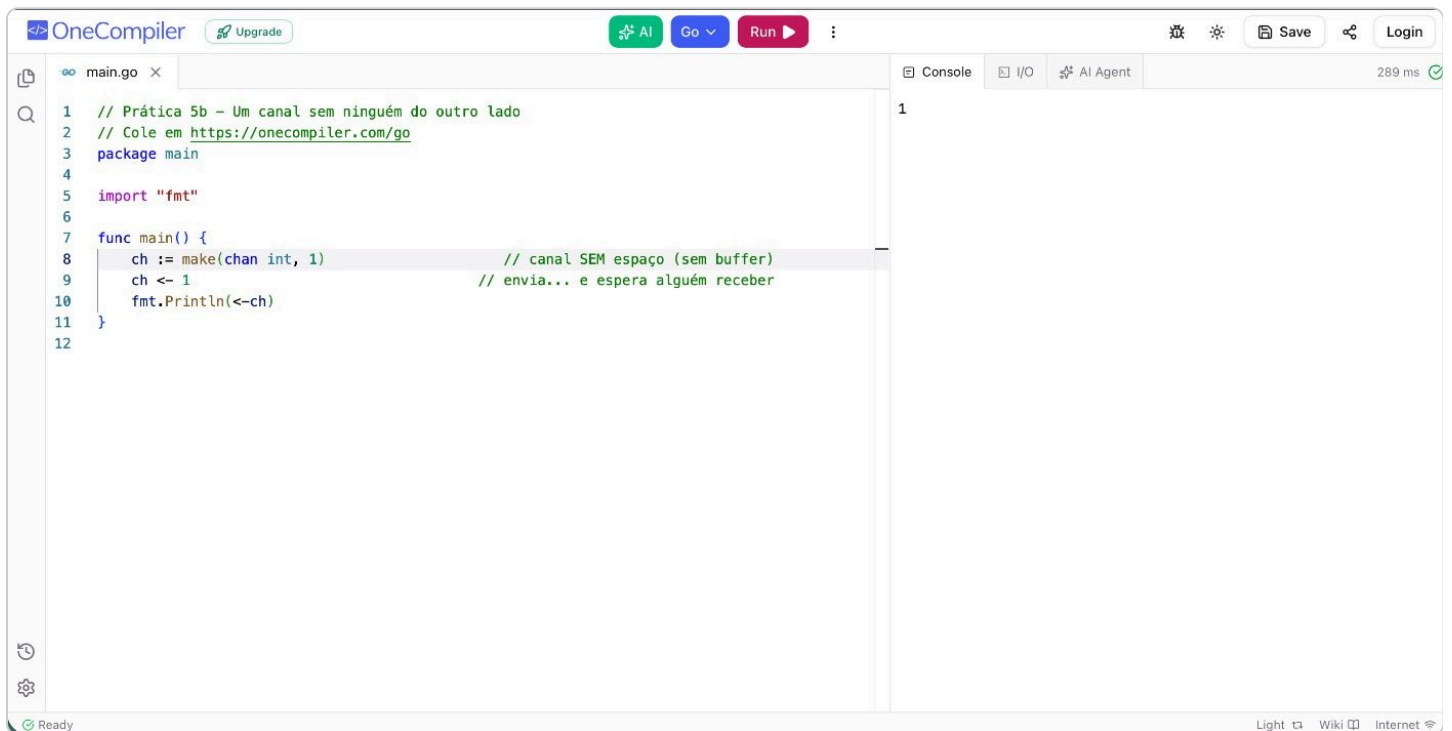
não respondido



The screenshot shows the OneCompiler interface with a Go file named `main.go`. The code is as follows:

```
1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int)           // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }
12
```

The console on the right displays a fatal error: `fatal error: all goroutines are asleep - deadlock!`. Below the error, it shows the stack trace for `goroutine 1 [chan send]:`, indicating the program is stuck at line 9.



The screenshot shows the same OneCompiler interface, but the code has been modified to include a buffer of size 1 in the channel creation:

```
1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int, 1)       // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }
12
```

The console on the right now shows a successful execution with the output `1` and a completion time of `289 ms`.

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Porque a mensagem com o trabalho a ser executado não chega ao trabalhador, e sem essa recepção ele fica parado esperando, o que leva ao deadlock.

Prática 6: O compilador como guarda

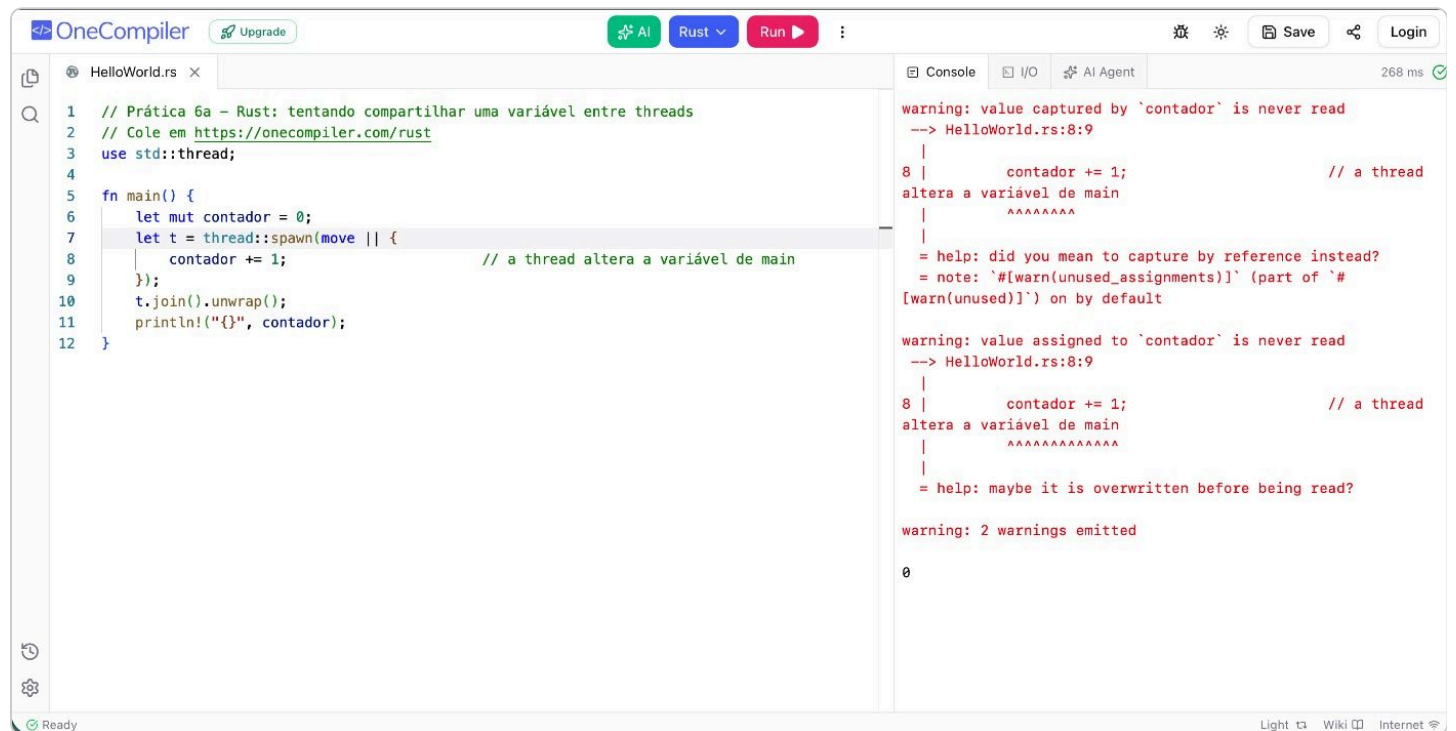
Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

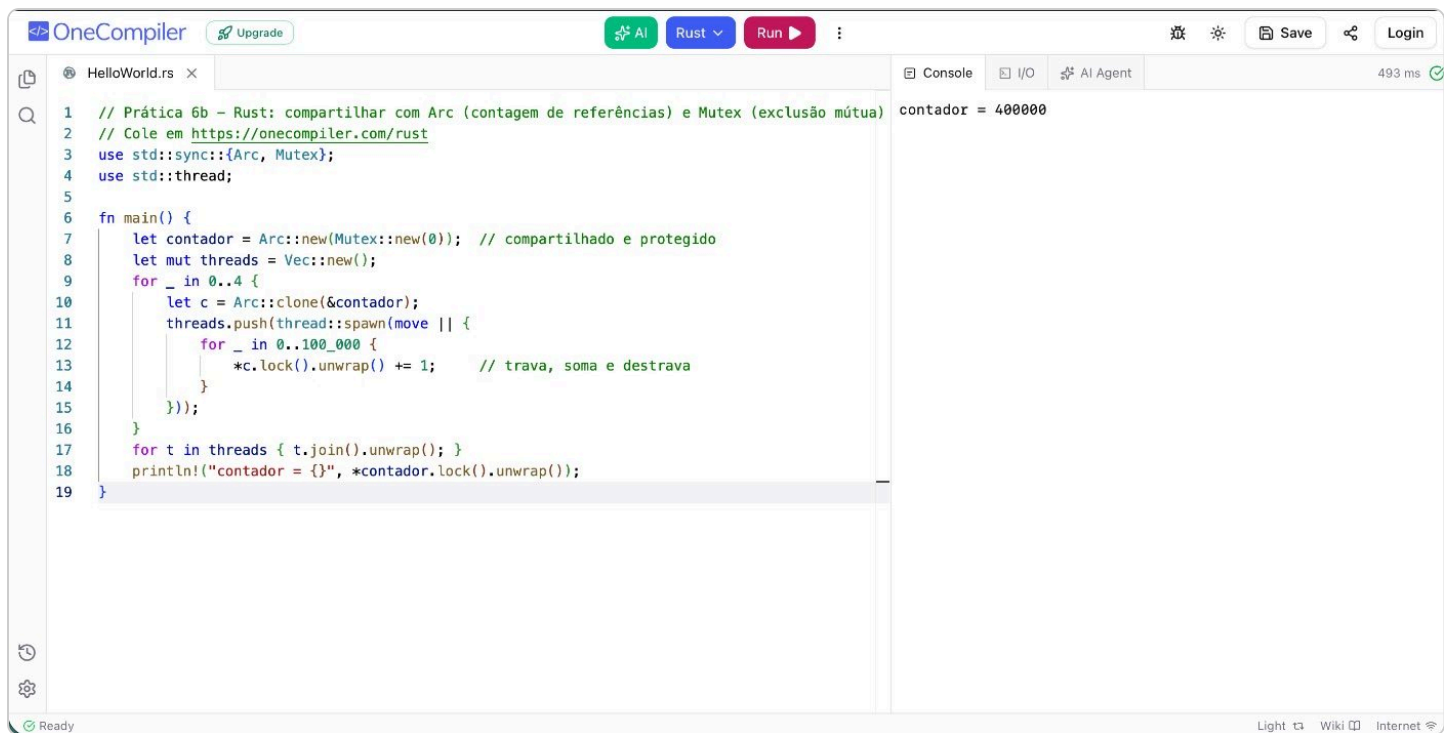
Como é um mutex, há exclusão mútua: só uma thread por vez acessa a seção crítica, e por isso o valor somado fica correto.

Saídas (o erro e o resultado com move)

Saída do programa

9/12





```
1 // Prática 6b - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10         let c = Arc::clone(&contador);
11         threads.push(thread::spawn(move || {
12             for _ in 0..100_000 {
13                 *c.lock().unwrap() += 1; // trava, soma e destrava
14             }
15         }));
16     }
17     for t in threads { t.join().unwrap(); }
18     println!("contador = {}", *contador.lock().unwrap());
19 }
```

contador = 400000

493 ms

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Porque o que é incrementado é a cópia da variável dentro da thread, enquanto a variável original continua zerada.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

O erro só apareceu em produção. A vantagem é a garantia, já na compilação, de que não haverá erro de concorrência, e o custo é um código mais complexo.

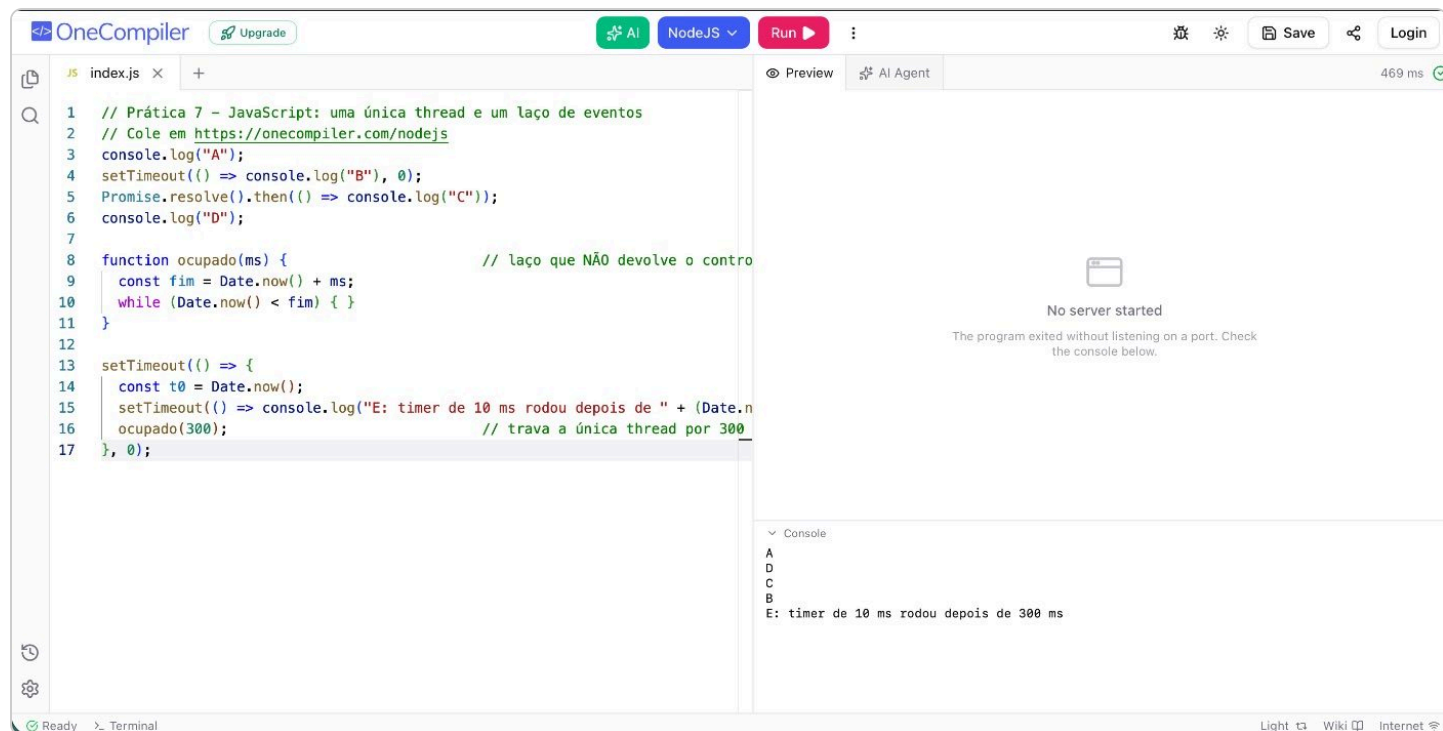
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A B D C

Saída do programa

não respondido



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A, D, C, B. Isso acontece porque o timeout (mesmo com tempo zerado) e a resolução da promise só são executados depois do código síncrono, então o console.log que vem logo abaixo é impresso antes deles.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque a execução da função ocupado bloqueou a impressão de E, já que o Node.js roda em uma única thread. Isso significa que um servidor Node.js com muitos usuários pode demorar para retornar os dados, pois fica travado na execução de outra tarefa.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

O mesmo lugar pode ser vendido para mais de uma pessoa. Uma solução seria fazer uma validação no momento do pagamento, verificando se aquele lugar já não foi comprado por outra pessoa.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O Go usa troca de mensagens entre goroutines (por meio de canais), em vez de memória compartilhada.